KODE MIDDLE MILE

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax', 'Kijang']

daerah = ['Barabai', 'Batu Licin', 'Rantau']

jarak = {'Barabai': 1046.393, 'Batu Licin': 924.619, 'Rantau': 179.485}

biaya_bbm = {

   'CDD': {'Barabai': 246238.2, 'Batu Licin': 345288.7, 'Rantau': 160248.8},

   'CDE': {'Barabai': 140707.5429, 'Batu Licin': 197307.8286, 'Rantau': 91570.74286},

   'GrandMax': {'Barabai': 181057.5, 'Batu Licin': 253888.75, 'Rantau': 117830},

   'Kijang' :{'Barabai': 144846, 'Batu Licin': 203111, 'Rantau': 94264}

}

biaya_kendaraan = {'CDD': 13853970, 'CDE': 6560843.333, 'GrandMax':6560843.333, 'Kijang':6560843.333}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120, 'Kijang' : 80}

demand = {'Barabai': 12496, 'Batu Licin': 7766, 'Rantau': 4287}

kapasitas_max_per_trip = 5280


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d] * 2+ biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 46 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")
```

```
# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 46
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
```

```
            total_unit = trips * kapasitas_kendaraan[k]
            biaya = biaya_bbm[k][d] * trips
            total_bbm += biaya
            print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")
from gurobipy import Model, GRB


# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax']

daerah = ['Kupang', 'Maumere', 'Sumbawa', 'Bima']

jarak = {'Kupang': 1046.393, 'Maumere': 924.619, 'Sumbawa': 179.485, 'Bima':440.971}

biaya_bbm = {

   'CDD': {'Kupang': 1778868.1, 'Maumere': 1571852.3, 'Sumbawa': 305124.5, 'Bima':749650.7},

   'CDE': {'Kupang': 1016496.057, 'Maumere': 898201.3143, 'Sumbawa': 174356.8571, 'Bima':428371.8286},

   'GrandMax': {'Kupang': 1307991.25, 'Maumere': 1155773.75, 'Sumbawa': 224356.25, 'Bima':551213.75}

}

biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120}

demand = {'Kupang': 8540, 'Maumere': 2709, 'Sumbawa': 3519, 'Bima':5783}

kapasitas_max_per_trip = 1600


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d]*2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 8 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()


# ------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
```

```
lp_model = m.relax()
lp_model.optimize()


if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 8
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
```

```
            biaya = biaya_bbm[k][d] * trips
            total_bbm += biaya
            print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")


# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax', 'Kijang']

daerah = ['Bontang-Sangata']

jarak = {'Bontang-Sangata': 59.200}

biaya_bbm = {

   'CDD': {'Bontang-Sangata': 100640},

   'CDE': {'Bontang-Sangata': 57508.57143},

   'GrandMax': {'Bontang-Sangata': 74000},

   'Kijang':{'Bontang-Sangata': 59200},

}

biaya_kendaraan = {'CDD': 13853970, 'CDE': 6560843.333, 'GrandMax':6560843.333, 'Kijang':6560843.333}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120, 'Kijang':80}

demand = {'Bontang-Sangata':1330}

kapasitas_max_per_trip = 5280


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 158 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)
```

```
# Optimasi
m.optimize()

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 158
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
```

```
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
                total_bbm += biaya
                print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax', 'Kijang']

daerah = ['Ampah']

jarak = {'Ampah': 59.200}

biaya_bbm = {

   'CDD': {'Ampah': 583256.4},

   'CDE': {'Ampah': 333289.3714},

   'GrandMax': {'Ampah': 428865},

   'Kijang':{'Ampah': 343092},

}

biaya_kendaraan = {'CDD': 13853970, 'CDE': 6560843.333, 'GrandMax':6560843.333, 'Kijang':6560843.333}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120, 'Kijang':80}

demand = {'Ampah':4623}

kapasitas_max_per_trip = 1040


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 27 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)
```

```
# Optimasi
m.optimize()

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 27
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
```

```
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
                total_bbm += biaya
                print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax', 'Kijang']

daerah = ['Ketapang', 'Singkawang', 'Sintang']

jarak = {'Ketapang': 352.400, 'Singkawang': 151.000, 'Sintang': 321.000}

biaya_bbm = {

   'CDD': {'Ketapang': 599080, 'Singkawang': 256700, 'Sintang': 545700},

   'CDE': {'Ketapang': 342331.4286, 'Singkawang': 146685.7143, 'Sintang': 311828.5714},

   'GrandMax': {'Ketapang': 440500, 'Singkawang': 188750, 'Sintang': 401250},

   'Kijang' : {'Ketapang': 352400, 'Singkawang': 151000, 'Sintang': 321000}

}

biaya_kendaraan = {'CDD': 13853970, 'CDE': 6560843.333, 'GrandMax':6560843.333, 'Kijang':6560843.333}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120, 'Kijang':80}

demand = {'Ketapang': 3313, 'Singkawang': 3964, 'Sintang': 2845}

kapasitas_max_per_trip = 5280


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d]*2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 26 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()


# ------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")
```

```
# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()


if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 26
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
```

```
            total_unit = trips * kapasitas_kendaraan[k]
            biaya = biaya_bbm[k][d] * trips
            total_bbm += biaya
            print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")


# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax', 'Kijang']

daerah = ['Pangkalanbun']

jarak = {'Pangkalanbun': 212.077}

biaya_bbm = {

   'CDD': {'Pangkalanbun': 1778868.1},

   'CDE': {'Pangkalanbun': 1016496.057},

   'GrandMax': {'Pangkalanbun': 1307991.25},

   'Kijang':{'Pangkalanbun': 1046393},

}

biaya_kendaraan = {'CDD': 13853970, 'CDE': 6560843.333, 'GrandMax':6560843.333, 'Kijang':6560843.333}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120, 'Kijang':80}

demand = {'Pangkalanbun':6285}

kapasitas_max_per_trip = 1000


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 44 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)
```

```
# Optimasi
m.optimize()

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 44
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
```

```
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
                total_bbm += biaya
                print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax']

daerah = ['Kolaka', 'Unaaha', 'Bau-Bau']

jarak = {'Kolaka': 154.220, 'Unaaha': 13.780, 'Bau-Bau': 275.580}

biaya_bbm = {

   'CDD': {'Kolaka': 262174, 'Unaaha': 23426, 'Bau-Bau': 468486},

   'CDE': {'Kolaka': 149813.7143, 'Unaaha': 13386.28571, 'Bau-Bau': 267706.2857},

   'GrandMax': {'Kolaka': 192775, 'Unaaha': 17225, 'Bau-Bau': 344475},

}

biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120, 'Kijang':80}

demand = {'Kolaka': 3579, 'Unaaha': 2181, 'Bau-Bau': 2724}

kapasitas_max_per_trip = 1640


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d]*2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 33 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()


# ------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
```

```
lp_model = m.relax()
lp_model.optimize()


if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 33
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
```

```
            biaya = biaya_bbm[k][d] * trips
            total_bbm += biaya
            print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")


# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax']

daerah = ['Amurang', 'Bitung', 'Kotamobagu', 'Tomohon', 'Gorontalo']

jarak = {

    'Amurang': 55.400,

    'Bitung': 47.800,

    'Kotamobagu': 170.450,

    'Tomohon': 35.190,

    'Gorontalo': 396.910

}

biaya_bbm = {

    'CDD': {

        'Amurang': 94180,

        'Bitung': 81260,

        'Kotamobagu': 289765,

        'Tomohon': 59823,

        'Gorontalo': 674747

    },

    'CDE': {

        'Amurang': 53817.14286,

        'Bitung': 46434.28571,

        'Kotamobagu': 165580,

        'Tomohon': 34184.57143,

        'Gorontalo': 385569.7143

    },

    'GrandMax': {
```

```
        'Amurang': 69250,
        'Bitung': 59750,
        'Kotamobagu': 213062.5,
        'Tomohon': 43987.5,
        'Gorontalo': 496137.5
    }
}

biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}
kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120}
# Updated demand data
demand = {
    'Amurang': 1729,
    'Bitung': 2269,
    'Kotamobagu': 3692,
    'Tomohon': 2088,
    'Gorontalo': 8017
}

kapasitas_max_per_trip = 3400

# Model
m = Model("Vehicle_Optimization")

# Variabel keputusan
x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan
t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip
muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)

# Fungsi objektif: total biaya (kendaraan + bbm)
m.setObjective(quicksum(
    biaya_bbm[k][d] * t[k, d]*2 + biaya_kendaraan[k] * x[k, d]
    for k in kendaraan for d in daerah
), GRB.MINIMIZE)

# Kendala pemenuhan demand tiap daerah
```

```
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 23 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()


# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
```

```
lp_model.optimize()


if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 23
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
```

```
            total_bbm += biaya
            print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax']

daerah = ['Parigi', 'Luwuk', 'Mamuju', 'Poso']

jarak = {

    'Parigi': 71.720,

    'Luwuk': 589.500,

    'Mamuju': 406.820,

    'Poso': 206.670

}


biaya_bbm = {

    'CDD': {

        'Parigi': 121924,

        'Luwuk': 1002150,

        'Mamuju': 691594,

        'Poso': 351339

    },

    'CDE': {

        'Parigi': 69670.85714,

        'Luwuk': 572657.1429,

        'Mamuju': 395196.5714,

        'Poso': 200765.1429

    },

    'GrandMax': {

        'Parigi': 89650,
```

```
        'Luwuk': 736875,
        'Mamuju': 508525,
        'Poso': 258337.5
    }
}

biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}
kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120}
# Updated demand data
demand = {
   'Parigi': 1223,
    'Luwuk': 9643,
    'Mamuju': 2902,
    'Poso': 2016
}

kapasitas_max_per_trip = 1560

# Model
m = Model("Vehicle_Optimization")

# Variabel keputusan
x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan
t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip
muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)

# Fungsi objektif: total biaya (kendaraan + bbm)
m.setObjective(quicksum(
    biaya_bbm[k][d] * t[k, d]*2 + biaya_kendaraan[k] * x[k, d]
    for k in kendaraan for d in daerah
), GRB.MINIMIZE)

# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")
```

```
# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 15 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()


# -------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# -------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()
```

```
if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 15
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
                total_bbm += biaya
                print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")
```

```
    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")


# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
!pip install gurobipy
!mkdir -p /root/.gurobi

import gurobipy as gp

import pandas as pd
from gurobipy import Model, GRB, quicksum

# Data
kendaraan = ['CDD', 'CDE', 'GrandMax', ]
daerah = ['Belopa', 'Mangkutana', 'Toraja']
jarak = {
    'Belopa': 49.980,
    'Mangkutana': 138.450,
    'Toraja': 72.180
}

biaya_bbm = {
    'CDD': {
        'Belopa': 84966,
```

```
        'Mangkutana': 235365,
        'Toraja': 122706
    },
    'CDE': {
        'Belopa': 48552,
        'Mangkutana': 134494.2857,
        'Toraja': 70117.71429
    },
    'GrandMax': {
        'Belopa': 62475,
        'Mangkutana': 173062.5,
        'Toraja': 90225
    }
}

biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}
kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120}
demand = {
    'Belopa': 1924,
    'Mangkutana': 4425,
    'Toraja': 3785
}

kapasitas_max_per_trip = 1640

# Model
m = Model("Vehicle_Optimization")

# Variabel keputusan
x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan
t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")       # Jumlah trip
muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)

# Fungsi objektif: total biaya (kendaraan + bbm)
m.setObjective(quicksum(
    biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]
```

```
    for k in kendaraan for d in daerah
), GRB.MINIMIZE)

# -------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# -------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 67 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
```

```
    m.addConstr(t[k, d] >= 0)
    m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
  print("\n===== TABEL DISTRIBUSI TRIP =====\n")
  max_trip = 67
  print(f"{'JenisKendaraan/Trip':<20}", end="")
  for i in range(1, max_trip + 1):
    print(f"{i:^8}", end="")
  print()

  trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
  for k in kendaraan:
    for d in daerah:
      trips = int(t[k, d].x)
      for i in range(trips):
        if i < max_trip:
          trip_map[k][i].append(d[0])  # d[0] = inisial daerah

  for k in kendaraan:
    print(f"{k:<20}", end="")
    for i in range(max_trip):
```

```
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
                total_bbm += biaya
                print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax' ]

daerah = ['Barru', 'Polman', 'Sengkang']

jarak = {

   'Barru': 62.410,

   'Polman': 103.540,

   'Sengkang': 80.400

}


biaya_bbm = {

   'CDD': {

      'Barru': 106097,

      'Polman': 176018,

      'Sengkang': 136680

   },

   'CDE': {

      'Barru': 60626.85714,

      'Polman': 100581.7143,

      'Sengkang': 78102.85714

   },

   'GrandMax': {

      'Barru': 78012.5,

      'Polman': 129425,

      'Sengkang': 100500
```

```
    }
}


biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}
kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120}
demand = {
    'Barru': 967,
    'Polman': 3422,
    'Sengkang': 3554
}

kapasitas_max_per_trip = 1480

# Model
m = Model("Vehicle_Optimization")

# Variabel keputusan
x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan
t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip
muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)

# Fungsi objektif: total biaya (kendaraan + bbm)
m.setObjective(quicksum(
    biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]
    for k in kendaraan for d in daerah
), GRB.MINIMIZE)

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
```

```
lp_model = m.relax()

lp_model.optimize()


# Kendala pemenuhan demand tiap daerah

for d in daerah:

    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")


# Kendala maksimum kapasitas per trip

for k in kendaraan:

    for d in daerah:

        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")


# Kendala maksimum 8 trip per kendaraan

for k in kendaraan:

    for d in daerah:

        m.addConstr(t[k, d] <= 90 * x[k, d], f"MaxTrips_{k}_{d}")


# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)

for k in kendaraan:

    for d in daerah:

        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")


# Kendala nilai non-negatif

for k in kendaraan:

    for d in daerah:

        m.addConstr(t[k, d] >= 0)

        m.addConstr(muatan[k, d] >= 0)


# Optimasi

m.optimize()


# -------------------------------------------

# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price

# -------------------------------------------

print("\n==============================")

print(">> LP Relaxation Analysis")
```

```
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 90
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
```

```
        trips = int(t[k, d].x)
        if trips > 0:
            total_unit = trips * kapasitas_kendaraan[k]
            biaya = biaya_bbm[k][d] * trips
            total_bbm += biaya
            print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax']

daerah = ['Penajam-Grogot']

jarak = {'Penajam-Grogot': 161.958}

biaya_bbm = {

   'CDD': {'Penajam-Grogot': 275328.6},

   'CDE': {'Penajam-Grogot': 157330.6286},

   'GrandMax': {'Penajam-Grogot': 202447.5},

   'Kijang':{'Penajam-Grogot': 161958},

}

biaya_kendaraan = {'CDD': 13853970, 'CDE': 6560843.333, 'GrandMax':6560843.333, 'Kijang':6560843.333}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120, 'Kijang':80}

demand = {'Penajam-Grogot':4675}

kapasitas_max_per_trip = 2880


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 57 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()


# ------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")
```

```
# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 57
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
```

```
            total_unit = trips * kapasitas_kendaraan[k]
            biaya = biaya_bbm[k][d] * trips
            total_bbm += biaya
            print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")

!pip install gurobipy
!mkdir -p /root/.gurobi

import gurobipy as gp

import pandas as pd
from gurobipy import Model, GRB, quicksum

# Data
kendaraan = ['CDD', 'CDE', 'GrandMax' ]
daerah = ['Bulukumba', 'Gowa', 'Jeneponto', 'Pangkep', 'Takalar', 'Bone', 'Palopo', 'Pare-Pare', 'Sinjai']
jarak = {
    'Bulukumba': 200.000,
    'Gowa': 16.370,
    'Jeneponto': 97.420,
```

```
    'Pangkep': 55.230,
    'Takalar': 41.330,
    'Bone': 194.850,
    'Palopo': 375.380,
    'Pare-Pare': 162.640,
    'Sinjai': 224.380
}

biaya_bbm = {
    'CDD': {
        'Bulukumba': 340000,
        'Gowa': 27829,
        'Jeneponto': 165614,
        'Pangkep': 93891,
        'Takalar': 70261,
        'Bone': 331245,
        'Palopo': 638146,
        'Pare-Pare': 276488,
        'Sinjai': 381446
    },
    'CDE': {
        'Bulukumba': 194285.7143,
        'Gowa': 15902.28571,
        'Jeneponto': 94636.57143,
        'Pangkep': 53652,
        'Takalar': 40149.14286,
        'Bone': 189282.8571,
        'Palopo': 364654.8571,
        'Pare-Pare': 157993.1429,
        'Sinjai': 217969.1429
    },
    'GrandMax': {
        'Bulukumba': 250000,
        'Gowa': 20462.5,
        'Jeneponto': 121775,
        'Pangkep': 69037.5,
```

```
        'Takalar': 51662.5,
        'Bone': 243562.5,
        'Palopo': 469225,
        'Pare-Pare': 203300,
        'Sinjai': 280475
    }
}




biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}
kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120}
demand = {
    'Bulukumba': 2549,
    'Gowa': 11369,
    'Jeneponto': 2561,
    'Pangkep': 1826,
    'Takalar': 1978,
    'Bone': 2871,
    'Palopo': 3259,
    'Pare-Pare': 5309,
    'Sinjai': 1169
}

kapasitas_max_per_trip = 2920

# Model
m = Model("Vehicle_Optimization")

# Variabel keputusan
x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan
t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip
muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)

# Fungsi objektif: total biaya (kendaraan + bbm)
m.setObjective(quicksum(
```

```
    biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]
    for k in kendaraan for d in daerah
), GRB.MINIMIZE)

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 26 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
```

```
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)

# Optimasi
m.optimize()

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 26
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
```

```
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
                total_bbm += biaya
                print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```

```
!pip install gurobipy

!mkdir -p /root/.gurobi


import gurobipy as gp


import pandas as pd

from gurobipy import Model, GRB, quicksum


# Data

kendaraan = ['CDD', 'CDE', 'GrandMax', ]

daerah = ['Marisa']

jarak = {'Marisa': 134.660}

biaya_bbm = {

   'CDD': {'Marisa': 228922},

   'CDE': {'Marisa': 130812.5714},

   'GrandMax': {'Marisa': 168325},

}


biaya_kendaraan = {'CDD': 6900000, 'CDE': 5500000, 'GrandMax': 3500000}

kapasitas_kendaraan = {'CDD': 280, 'CDE': 200, 'GrandMax': 120}

demand = {'Marisa':2672}

kapasitas_max_per_trip =1120


# Model

m = Model("Vehicle_Optimization")


# Variabel keputusan

x = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Vehicles")    # Jumlah kendaraan

t = m.addVars(kendaraan, daerah, vtype=GRB.INTEGER, name="Trips")      # Jumlah trip

muatan = m.addVars(kendaraan, daerah, vtype=GRB.CONTINUOUS, name="Load") # Jumlah muatan (kg)


# Fungsi objektif: total biaya (kendaraan + bbm)

m.setObjective(quicksum(

   biaya_bbm[k][d] * t[k, d] *2 + biaya_kendaraan[k] * x[k, d]

   for k in kendaraan for d in daerah

), GRB.MINIMIZE)
```

```
# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

# Kendala pemenuhan demand tiap daerah
for d in daerah:
    m.addConstr(quicksum(muatan[k, d] for k in kendaraan) >= demand[d], f"Demand_{d}")

# Kendala maksimum kapasitas per trip
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_max_per_trip * t[k, d], f"MaxPerTrip_{k}_{d}")

# Kendala maksimum 8 trip per kendaraan
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] <= 69 * x[k, d], f"MaxTrips_{k}_{d}")

# Kendala kapasitas maksimum kendaraan tiap trip (tidak bisa bawa lebih dari kapasitasnya)
for k in kendaraan:
    for d in daerah:
        m.addConstr(muatan[k, d] <= kapasitas_kendaraan[k] * t[k, d], f"VehicleCapacity_{k}_{d}")

# Kendala nilai non-negatif
for k in kendaraan:
    for d in daerah:
        m.addConstr(t[k, d] >= 0)
        m.addConstr(muatan[k, d] >= 0)
```

```
# Optimasi
m.optimize()

# ---------------------------------------------
# Lanjutkan ke LP Relaxation untuk dapatkan RC & Shadow Price
# ---------------------------------------------
print("\n==============================")
print(">> LP Relaxation Analysis")
print("==============================")

# Relax model ke LP (integer jadi continuous)
lp_model = m.relax()
lp_model.optimize()

if m.status == GRB.OPTIMAL:
    print("\n===== TABEL DISTRIBUSI TRIP =====\n")
    max_trip = 69
    print(f"{'JenisKendaraan/Trip':<20}", end="")
    for i in range(1, max_trip + 1):
        print(f"{i:^8}", end="")
    print()

    trip_map = {k: [[] for _ in range(max_trip)] for k in kendaraan}
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            for i in range(trips):
                if i < max_trip:
                    trip_map[k][i].append(d[0])  # d[0] = inisial daerah

    for k in kendaraan:
        print(f"{k:<20}", end="")
        for i in range(max_trip):
            tujuan = "".join(trip_map[k][i])
            print(f"{tujuan:<8}", end="")
```

```
        print()
    print("\nHasil Optimasi:")
    for k in kendaraan:
        for d in daerah:
            print(f"{k} ke {d}: Vehicles = {x[k, d].x:.0f}, Trips = {t[k, d].x:.0f}, Muatan = {muatan[k, d].x:.2f} box")

    total_bbm = 0
    for k in kendaraan:
        for d in daerah:
            trips = int(t[k, d].x)
            if trips > 0:
                total_unit = trips * kapasitas_kendaraan[k]
                biaya = biaya_bbm[k][d] * trips
                total_bbm += biaya
                print(f"Kendaraan {k} mengirim {total_unit:.0f} box ke {d} dalam {trips} trip (Biaya BBM: Rp{biaya:,.0f})")

    print(f"\nTotal Biaya BBM: Rp{total_bbm:,.0f}")
    total_cost = m.objVal

else:
    print("Optimal solution not found.")

# Variabel dan Reduced Cost
print("\n>> Variabel dan Reduced Cost (LP Relaxation):")
for v in lp_model.getVars():
    print(f"{v.VarName:25} Value = {v.X:6.2f}, Reduced Cost = {v.RC:6.2f}")

# Kendala dan Shadow Price
print("\n>> Kendala dan Shadow Price (Dual Values):")
for c in lp_model.getConstrs():
    print(f"{c.ConstrName:25} Slack = {c.Slack:6.2f}, Shadow Price = {c.Pi:6.2f}")
```